ByteDMD × RMM — proof that live-byte cost scales as $N^3 \log N$

Prompt. Prove that RMM under ByteDMD cost (counting live bytes only) scales as $N^3 \log N$.

To rigorously prove that the ByteDMD cost of standard Recursive Matrix Multiplication (RMM) scales as $\Theta(N^3 \log N)$, we must analyze the algorithm's memory access patterns through the mechanics of ByteDMD's LRU stack-distance penalty ($\sqrt{d}$) and its aggressive liveness compaction.

We can mathematically prove this scaling using two elegant approaches: establishing the algorithm's data-movement recurrence relation, and using an integral identity that links ByteDMD to standard cache complexity.

Part 1: Why "live bytes only" is the crucial clamping rule

In an $N \times N$ RMM, the algorithm allocates intermediate blocks ($M_1 \ldots M_8$) at every step of the recursion. Over the algorithm's lifecycle, this creates $\Theta(N^3)$ temporary elements.

If dead variables were left on the stack, inputs accessed late in the execution (like A_{22}) would be pushed under all these $\Theta(N^3)$ temporaries. Fetching them would incur a disastrous $\sqrt{\Theta(N^3)} = \Theta(N^{1.5})$ penalty per element, inflating the overall scaling to $\Theta(N^{3.5})$.

However, ByteDMD strictly tracks **live bytes only**. The moment an intermediate result is consumed (e.g., M_1 and M_2 vaporize after computing $C_{11} = M_1 + M_2$), they are evicted. This optimal garbage collection ensures the stack footprint is strictly clamped to the active working set.

- **The bound.** At any recursion level operating on $K \times K$ blocks, the stack contains only the inputs, the output, and a constant number of active temporaries. The maximum stack depth $d_{\max}$ is therefore strictly bounded by $\Theta(K^2)$.
- **The penalty.** The maximum cost to read any live element during a size-K subproblem is $\sqrt{\Theta(K^2)} = \Theta(K)$.

Part 2: The proof by recurrence relation

RMM computes $C = A \times B$ by making 8 recursive calls of size $(N/2) \times (N/2)$ and performing 4 matrix additions. Let $T(N)$ be the total ByteDMD cost. At any recursive level, we incur local data-movement overhead $f(N)$ from two sources.

1. The matrix-addition penalty $\Theta(N^3)$

Consider the addition $C_{11} = M_1 + M_2$.

- M_1 is computed first.
- Next, M_2 executes, allocating its own temporaries and fetching its inputs (A_{12}, B_{21}). This brings $\Theta(N^2)$ elements to the top of the stack.
- Because M_1 is idle during M_2's execution, the LRU policy pushes M_1 down to a depth of $\Omega(N^2)$.
- When the addition executes, reading the $N^2/4$ elements of M_1 from depth $\Omega(N^2)$ incurs a cost of $\sqrt{\Omega(N^2)} = \Theta(N)$ per element.
- Across the 4 matrix additions, the total ByteDMD routing cost is $4 \times \frac{N^2}{4} \times \Theta(N) = \Theta(N^3)$.

2. The context-switch penalty $\Theta(N^3)$

Between recursive calls (e.g., pivoting from $M_1 = A_{11}B_{11}$ to $M_3 = A_{11}B_{12}$), the active working set shifts. Intervening subproblems push required inputs down the stack. Pulling an $N/2 \times N/2$ block back to the top of the stack from depth $\Theta(N^2)$ requires another $\Theta(N^2) \times \Theta(N) = \Theta(N^3)$ in data movement.

3. Applying the Master Theorem

Once inputs are fetched to the top, the subproblem operates locally. Because it continuously hits the top of the stack, its execution perfectly mirrors a standalone smaller problem, costing $T(N/2)$.

We get the divide-and-conquer recurrence

$$T(N) = 8\,T(N/2) + \Theta(N^3).$$

Applying the Master Theorem ($T(N) = a\,T(N/b) + \Theta(N^c)$):

- $a = 8$ (sub-problems)
- $b = 2$ (shrink factor)
- $c = 3$ (overhead scaling exponent)

Since $a = b^c$ (i.e., $8 = 2^3$), we fall precisely into **Case 2** of the Master Theorem. The geometric proliferation of sub-branches exactly balances the local data-movement overhead. The work distributes evenly across all $\log_2 N$ levels of the tree, yielding

$$T(N) = \Theta(N^3 \log N). \qquad \blacksquare$$

Part 3: The proof by continuous cache-miss integral

We can cross-verify this result using a beautiful integral identity linking the ByteDMD $\sqrt{d}$ penalty to the standard I/O cache complexity $Q(M)$ for an ideal LRU cache of size M.

An element read at depth d_i represents a cache miss in an LRU cache of size M if and only if $d_i > M$. We can rewrite the ByteDMD cost square root as an integral:

$$\sum_{\text{reads}} \sqrt{d_i} \;=\; \sum_i \int_0^{d_i} \frac{1}{2\sqrt{M}}\, dM.$$

By interchanging the sum and the integral, we get an indicator function $\mathbb{1}(d_i > M)$, which exactly equals the cache-miss complexity $Q(M)$:

$$\text{Total Cost} \;=\; \int_0^{\infty} \frac{\sum_i \mathbb{1}(d_i > M)}{2\sqrt{M}}\, dM \;=\; \frac{1}{2}\int_0^{\infty} \frac{Q(M)}{\sqrt{M}}\, dM.$$

Integrating the RMM bound

It is a well-known theorem (Hong & Kung, 1981) that the cache-oblivious complexity of standard RMM is $Q(M) = \Theta(N^3/\sqrt{M})$.

Furthermore, because ByteDMD utilizes eager initialization (inputs are pre-loaded) and liveness clamping (temporaries are vaporized), the stack never exceeds $M_{\max} = \Theta(N^2)$. Thus, for any cache capacity $M > \Theta(N^2)$, the hit rate is 100% and $Q(M) = 0$.

Plugging this into our identity perfectly extracts the ByteDMD cost:

$$\text{Cost} \;=\; \Theta(1)\int_1^{\Theta(N^2)} \frac{\Theta(N^3/\sqrt{M})}{\sqrt{M}}\, dM \;=\; \Theta(N^3)\int_1^{\Theta(N^2)} \frac{1}{M}\, dM.$$

Evaluating $\int \frac{1}{M}\, dM = \ln M$:

$$\text{Cost} = \Theta(N^3)\Big[\ln M\Big]_1^{\Theta(N^2)} = \Theta\big(N^3 \ln(N^2)\big) = \Theta(N^3 \log N).$$

Bonus insight: why Strassen dominates RMM under ByteDMD

This mathematical framing cleanly reveals why Strassen's algorithm dominates standard RMM under the ByteDMD metric (as mentioned in the prompt's reports).

Strassen requires only **7** recursive sub-calls instead of 8, while retaining the exact same $\Theta(N^3)$ matrix-addition footprint overhead. Its recurrence becomes

$$T(N) = 7\,T(N/2) + \Theta(N^3).$$

Because $7 < 2^3$, Strassen falls into **Case 3** of the Master Theorem. The geometric decay of the sub-problems means the cost is overwhelmingly bound by the top-level overhead.

Under ByteDMD, Strassen's true theoretical advantage isn't scaling as $N^{2.81}$ — its spatial cost scales strictly as

$$T(N) = \Theta(N^3).$$

It overtakes naive RMM by mathematically stripping away the entire $\log N$ spatial-routing penalty.